Piecewise Clothoid Smoothing for GPS Trajectories

Module Documentation: `clothoid.py`

GPS Track Conditioning Pipeline

Contents

1 Introduction

The `clothoid.py` module fits **piecewise clothoid segments** (Euler spirals) to a Kalman-filtered vehicle trajectory. Clothoids are curves whose curvature changes linearly with arc length —the natural model for a vehicle driven at constant steering rate [1]. The module provides:

- Automatic segmentation of the trajectory based on curvature linearity
- Per-segment least-squares fitting of clothoid parameters
- Heading (θ) continuity enforcement at segment joints
- Curvature allowed to be discontinuous at joints (physically: abrupt steering)
- Analytic derivatives $d\kappa/ds$ for downstream dynamics

The clothoid smoother operates **downstream** of the Kalman filter and does **not** alter positions. It extracts a physically consistent curvature profile from the already-filtered trajectory.

1.1 Why Clothoids Instead of Bézier Curves?

Bézier curves and B-splines are the standard representation in computer graphics and CAD, and they share the same B-spline basis as the preprocessing spline used internally. Yet they are the **wrong** final model for vehicle trajectory work. The table below contrasts the two representations on the criteria that matter for automotive dynamics.

Criterion	Bézier / B-spline	Clothoid (Euler spiral)
Physical model	Curvature is a rational polynomial in the parameter. No known closed-form relationship to steering angle or yaw rate.	$\kappa(s) = \kappa_0 + \sigma \cdot s$ — exactly the curvature generated by a constant steering rate $\dot{\delta}$. Directly measurable from the bicycle model.
G2 continuity	Requires degree ≥ 5 and explicit endpoint constraints on control points. Knot insertion changes the global shape.	Guaranteed at every arc junction by `SolveG2` as a closed-form solve. No iterative optimisation needed.
Parameter meaning	Control points are off-curve. Adjusting one point changes curvature everywhere on the segment.	(κ_0, σ, L) have direct geometric meaning: entry curvature, curvature rate, and arc length. Each segment is independently interpretable.
Position reconstruction	Analytic (de Casteljau). No integration error.	Fresnel integrals — also analytic, no integration error, and directly analytic (generalized Fresnel integrals).
Compression	Degree-5 Bézier: 6 control points per segment (12 floats in 2D).	3 clothoid arcs per G2 interval: 5 floats (κ_0, σ, L) per arc = 15 floats per interval — but a single interval covers the same geometric complexity as multiple Bézier segments.
Lateral jerk	$d^2\kappa/dt^2$ is a high-degree polynomial; does not simplify for vehicle simulation.	$d\kappa/ds = \sigma$ (piecewise constant). Lateral jerk $\propto v^2\sigma$ is constant per arc and directly

Criterion	Bézier / B-spline	Clothoid (Euler spiral)
		available from the segment parameters.
Road design standard	Not used in road design standards.	ISO 6976 / German RASt: transition curves between circular arcs are specified as clothoids [2]. The model matches real road geometry by construction.

Table 1: Clothoids vs. Bézier curves for vehicle trajectory representation.

In summary: Bézier curves are optimal for rendering and shape design. Clothoids are optimal when the curve must be **physically interpretable**, **dynamically analysable**, and **compatible with road geometry standards** — all of which apply here.

1.2 Pipeline Position

Raw GPS (sample-and-hold)
↓ `processor.py`: S&H removal, interpolation
Interpolated GPS + wheel speed + steering angle (→ yaw rate via bicycle model)
↓ `kalman.py`: EKF + RTS smoother [3]
Kalman trajectory $(x, y, v, \theta, \omega)$
↓ **`clothoid.py`: piecewise clothoid fit**
Curvature profile (κ_0, σ, L) per segment

2 Mathematical Foundation

2.1 The Clothoid (Euler Spiral)

A **clothoid** is a plane curve whose curvature varies linearly with arc length s:

$$\kappa(s) = \kappa_0 + \sigma \cdot s \tag{1}$$

where:

- κ_0 — curvature at the segment start [1/m]
- σ — curvature rate (sharpness) [1/m²]
- s — arc length measured from segment start [m]

The parameter σ corresponds directly to constant steering rate: a driver turning the wheel at constant angular velocity produces exactly a clothoid path.

2.2 Heading Along a Clothoid

Integrating curvature gives the heading angle:

$$\theta(s) = \theta_0 + \kappa_0 s + \frac{1}{2}\sigma s^2 \tag{2}$$

This is a **quadratic** function of arc length — the defining property of an Euler spiral.

2.3 Position Along a Clothoid (Fresnel Integrals)

Position requires integrating the heading:

$$x(s) = x_0 + \int_0^s \cos(\theta(t))\,\mathrm{d}t \tag{3}$$

$$y(s) = y_0 + \int_0^s \sin(\theta(t))\,\mathrm{d}t \tag{4}$$

These integrals have no closed-form solution (they are generalized **Fresnel integrals**). The implementation evaluates them numerically via the trapezoidal rule with adaptive step size.

2.4 Segment Parameters

Each clothoid segment is fully described by 6 parameters:

Parameter	Symbol	Description
`x0`	x_0	Start position, East [m]
`y0`	y_0	Start position, North [m]
`theta0`	θ_0	Start heading [rad]
`kappa0`	κ_0	Start curvature [1/m]
`sigma`	σ	Curvature rate [1/m^2]
`length`	L	Arc length of segment [m]

Table 2: Parameters of a single `ClothoidSegment` (arc output of `G2ClothoidApproximator`).

Given these 6 numbers, the entire segment geometry is determined: heading via Equation 2, position via Equation 4, and curvature at any point via Equation 1.

3 G2-Continuous Clothoid Approximation

3.1 Overview

The `G2ClothoidApproximator` fits a G2-continuous chain of 3-arc clothoid intervals to the GPS reference directly. pipeline. Each interval between two support points is solved by `pyclothoids.SolveG2` (Bertolazzi-Frego), which finds 3 arcs satisfying position + tangent + curvature constraints at both endpoints. The curvature jump at every arc junction is machine-epsilon ($\approx 10^{-15}$ 1/m).

A greedy forward-search with bisection minimises the number of support points while keeping the max lateral error $\leq$ `position_tolerance_m`.

3.1.1 The Bertolazzi-Frego SolveG2 Algorithm

`SolveG2` was published by Enrico Bertolazzi and Marco Frego (University of Trento) [4] as a closed-form construction of a **3-arc clothoid spline** that interpolates two endpoints with prescribed position, tangent, and curvature — the minimal set of boundary data needed for G2 continuity. The Python binding is provided by the `pyclothoids` library [5].

3.1.1.1 Constraint Counting

A single clothoid arc has 5 free parameters: $(x_0, y_0, \theta_0, \kappa_0, \sigma)$ with length L fixed by the endpoint constraint. Requiring G2 at both ends of a single arc imposes 8 scalar constraints (position: 4, tangent: 2, curvature: 2) but a single arc has only 2 free parameters (κ_0, σ) once the boundary states are fixed — the system is severely over-determined.

Three arcs in series have $3 \times 2 = 6$ internal degrees of freedom (the $\left(\kappa_0^{(i)}, \sigma^{(i)}\right)$ of each arc) plus 3 arc lengths L_1, L_2, L_3 — giving 9 unknowns. The G2 constraints at the two **outer** endpoints consume 8 equations, and G1 continuity at the two **inner** junctions adds 4 more. Bertolazzi and Frego show that this $9 + 4 = 13$-dimensional system can be reduced to a **single scalar**

equation in one unknown via a change of variables, solved in 3–5 Newton iterations. The solution is unique under mild regularity conditions.

Boundary	Constraints	Enforced by
Start $(x_0, y_0, \theta_0, \kappa_0)$	4	Arc 1 initial state
Inner junction 1 (arc 1 $\rightarrow$ arc 2)	2 (G1)	Tangent continuity
Inner junction 2 (arc 2 $\rightarrow$ arc 3)	2 (G1)	Tangent continuity
End $(x_1, y_1, \theta_1, \kappa_1)$	4	Newton residual $\rightarrow 0$
Total	**12**	**9 unknowns + scalar root find**

Table 3: Constraint accounting for the 3-arc SolveG2 construction.

3.1.1.2 Why Three Arcs Fit Road Geometry Naturally

Road transition zones are conventionally designed as:

$$\text{straight} \rightarrow \text{clothoid (entry)} \rightarrow \text{circular arc (apex)} \rightarrow \text{clothoid (exit)} \rightarrow \text{straight} \tag{5}$$

A single `SolveG2` interval — **entry arc + central arc + exit arc** — maps directly onto this structure. The central arc absorbs the bulk of the bend; the two flanking arcs handle the curvature transition. This is not a coincidence: the Euler spiral was adopted in road design precisely because it models constant-steering-rate driving, and `SolveG2` reconstructs the same three-phase manoeuvre from the boundary data alone.

3.1.1.3 Why SolveG2 Fits This Application Particularly Well

Property	Significance for GPS track fitting
Exact G2 at boundaries	Curvature is continuous across every support point by construction. No post-hoc smoothing or constrained optimisation needed.
Closed-form (no iteration on positions)	Runtime is $O(1)$ per interval regardless of interval length. The greedy search calls `SolveG2` $O(\log N)$ times per support point — total cost is $O(N \log N)$.
Input matches available data	θ and κ at each support point come directly from the preprocessed spline — the same quantities `SolveG2` requires. No conversion or approximation is needed.
Physical 3-arc structure	Each interval models approach, apex, and exit — the same structure as a designed road curve. A single interval can span an entire motorway bend.
Machine-epsilon curvature jump	The residual $\lvert\Delta\kappa\rvert$ at junctions is $\approx 10^{-15}$ 1/m — limited only by IEEE 754 double precision. This is not an approximation; it follows from the algebraic construction.
`pyclothoids` binding	The `pyclothoids` library provides a direct Python binding to the Bertolazzi-Frego C++ implementation, with a well-tested API:

Property	Significance for GPS track fitting
	`SolveG2(x0,y0,θ0,κ0, x1,y1,θ1,κ1)` → list of 3 clothoids.

Table 4: Why `SolveG2` is the right primitive for G2-continuous GPS track approximation.

3.2 Best-Results Process Chain

The following end-to-end pipeline produces the tightest, most physically consistent clothoid chain from raw GPS data. Each step removes a specific error source; skipping any step degrades either accuracy or continuity.

Step	Tool / Parameter	Purpose
1. Kalman pre-filter	`GPSProcessor( smoothing='kalman')`	Removes S&H duplicates, interpolates GPS gaps, fuses wheel speed and steering-angle yaw rate (EKF + RTS). Produces a smooth $(x, y, v, \theta, \omega)$ state at full sample rate. This is the reference the clothoid chain must follow.
2. Deduplication	`dedup_threshold_m=1e-3`	Removes residual co-located GPS fixes ($\Delta s < 1$ mm) that would cluster B-spline knots and inject high-frequency curvature noise into step 3.
3. Chord-length parametrisation	`t = cumsum(hypot(dx, dy))`	Replaces index parametrisation. Ensures the smoothing budget is distributed proportionally to arc length rather than sample count — critical at variable speed.
4. Degree-5 smoothing spline	`spline_degree=5`, `gps_sigma_m`	Fits a quintic B-spline to $(x(t), y(t))$. Degree 5 is required for stable analytic second derivatives. `gps_sigma_m` controls the smoothing budget: larger values accept more deviation from the raw GPS in exchange for smoother κ.
5. Analytic $\kappa(s)$	$\kappa = (x'y'' - y'x'')/(x'^2 + y'^2)^{3/2}$	Curvature from spline derivatives — exact on the spline, no finite-difference noise. The zero-speed guard `_SPEED_SQ_MIN` prevents division by zero at very slow or stationary epochs.

Step	Tool / Parameter	Purpose
6. Moving-average smoothing	`kappa_smooth_window`	Suppresses residual spline-fitting noise on the $\kappa(s)$ profile. Window in samples: at `resample_spacing_m=0.5` a window of 51 corresponds to 25 m — wide enough to remove GPS-scale noise without blurring genuine curvature transitions.
7. Optional κ clip	`kappa_clip`	Clamps $\lvert\kappa\rvert$ to a physical maximum (e.g. 0.20 1/m = R=5 m). Use only when GPS artefacts produce physically impossible hairpins. Leave `None` on clean tracks.
8. Heading from κ	$\theta = \theta_0 + \int_0^s \kappa \, \mathrm{d}s$	Recomputes $\theta(s)$ by integrating the smoothed κ. Guarantees θ and κ are mutually consistent — preventing excess support points caused by the divergence between `atan2`-derived θ and smoothed κ.
9. Greedy SolveG2 fitting	`initial_step`, `position_tolerance_m`	A greedy forward search doubles the interval length until the max lateral error exceeds `position_tolerance_m`, then bisects to find the exact boundary. Each interval is solved by `pyclothoids.SolveG2` (closed-form, no iteration). Guarantees $\lvert\kappa\rvert$-jump $< 10^{-14}$ 1/m at every junction.
10. Dense error validation	`eval_spacing_m=0.3`	Error is measured against a densely sampled (0.3 m) clothoid chain, not the sparse reference grid. Prevents the "blind-spot bias" where errors between reference grid points go undetected.

Table 5: End-to-end pipeline for best clothoid fitting results. Steps 2–8 occur inside `_smooth_and_parametrize()`; step 9–10 inside `G2ClothoidApproximator.fit()`.

3.2.1 Recommended Parameter Values

For a clean Kalman-filtered track at 100–200 km/h:

```
G2ClothoidConfig(
    gps_sigma_m          = 0.5,  # Kalman pre-smoothed — trust positions
    spline_degree        = 5,    # required for analytic κ
    resample_spacing_m   = 0.5,  # fine grid; closes blind-spot bias
    kappa_smooth_window  = 51,   # ≈ 25 m at 0.5-m grid — removes GPS noise
    kappa_clip           = None, # None = clean track; 0.20 = GPS hairpins
    position_tolerance_m = 0.5,  # tightest; 0.3 m → ~3× more intervals
    eval_spacing_m       = 0.3,  # must be << resample_spacing_m
    initial_step         = 16,   # start (32 for long smooth tracks)
)
```

Increasing `gps_sigma_m` or `resample_spacing_m` reduces interval count at the cost of accuracy. Decreasing `position_tolerance_m` below 0.3 m gives diminishing returns because the residual error is dominated by GPS noise, not the clothoid model.

3.3 Reference Preprocessing

The input GPS positions are preprocessed in `_smooth_and_parametrize()` before B-spline fitting. Four steps are required for correct `SolveG2` behaviour:

Step	Implementation	Rationale
Deduplication	`keep = step_ds > dedup_threshold_m`	Removes S&H duplicate fixes (~7 % of high-rate GPS streams) that cluster spline knots and inject curvature noise
Chord-length parameter	`t = cumsum(hypot(dx, dy))`	Index parametrisation (`t = arange(n)`) distributes the smoothing budget unevenly at variable speed; chord length is a near-arc-length parameter matching the `splprep`-based reference
Analytic κ	$\kappa = (x'y'' - y'x'')/(x'^2 + y'^2)^{3/2}$	Spline 2nd derivatives x'', y'' are computed analytically — more accurate than `np.gradient(theta)` whose finite differences add noise after moving-average smoothing
Heading from κ	$\theta = \theta_0 + \int_0^s \kappa \, ds$	Guarantees θ and κ are mutually consistent. If θ comes from `atan2` and κ from the smoothed gradient, they diverge after the moving average, forcing excess knots

Table 6: Four preprocessing steps in `_smooth_and_parametrize()`. All four must be applied together.

3.4 Configuration

Parameter	Default	Description
`gps_sigma_m`	2.0 m	Spline smoothing: $s = n \cdot \sigma^2$

Parameter	Default	Description
`dedup_threshold_m`	1e-3 m	Min step between consecutive raw points; smaller = S&H duplicate
`spline_degree`	5	B-spline degree (5 required for stable analytic κ)
`resample_spacing_m`	2.0 m	Uniform arc-length grid spacing
`kappa_smooth_window`	11	Moving-average window for analytic κ (samples)
`kappa_clip`	None	Max $\|\kappa\|$ [1/m]; set 0.20 for GPS-hairpin artefacts
`position_tolerance_m`	0.5 m	Max lateral error per greedy interval
`eval_spacing_m`	0.3 m	Clothoid sampling density during error evaluation
`replace_positions`	False	Return reconstructed clothoid path instead of Kalman positions

Table 7: `G2ClothoidConfig` parameters (2026-06-27).

3.5 Performance

Track / Config	Intervals	Max error	Runtime
200 k samples, 3.86 km, `kappa_clip=None`	≈80	<0.50 m (100 %)	≈3 s
50 k samples (GPS artefact), `kappa_clip=0.20`	≈197	<0.50 m (100 %)	≈0.6 s

Table 8: G2 approximator benchmark. Max κ-jump at any junction $\approx 10^{-15}$ 1/m.

4 Module Architecture

4.1 Class Hierarchy

```
clothoid.py
├── ClothoidConfig          (dataclass: tuning parameters)
├── ClothoidResult          (dataclass: output container)
├── ClothoidSegment         (dataclass: per-segment parameters)
│   ├── smooth()            → ClothoidResult
│   ├── _segment_curvature()    (breakpoint detection)
│   ├── _fit_segments()         (independent LS per segment)
│   └── _build_curvature_profile()     (full-resolution κ)
│   ├── evaluate()          → (s, x, y, θ, κ)
│   ├── curvature_at(s)     → κ (piecewise linear)
│   ├── dkappa_ds(s)        → σ (piecewise constant)
│   ├── position_at(s)      → (x, y)
│   ├── support_vector()    → (u, k) analysis only
│   └── support_vector_dual() → (u, k) with jumps
├── G2ClothoidApproximator  (greedy knot placement, Bertolazzi-Frego SolveG2)
```

4.2 Data Flow

5 Practical Considerations

5.1 Low-Speed Pathology

At vehicle speeds below 20 km/h, GPS positions cluster and heading becomes unreliable. The clothoid fitter produces extreme curvatures ($|\kappa| > 1$ rad/m, corresponding to turn radii < 1 m) for these segments.

Mitigation: The heading-matched initialization uses reference heading changes $\Delta\theta_{\text{ref}}$ from the downsampled Kalman trajectory, which is already smoothed. This avoids pathological κ values

from noisy local position clusters. For truly stationary segments, κ_0 will be near zero (minimal heading change over zero distance → undefined, but length is also zero → segment contributes nothing to the chain).

5.2 Computation Cost

The dominant cost is the per-segment least-squares optimization. Typical timings for a 4 km track (200k input samples):

- Downsampling + segmentation: < 10 ms
- 200 segment fits (Levenberg–Marquardt): 70 ms
- Heading-matched θ-chain: 5 ms (single forward pass, no optimization)
- Full-resolution expansion: 5 ms
- **Total**: 90 ms (added to the 4.1 s Kalman runtime)

The `G2ClothoidApproximator` adds 0.6–3 s for full-track approximation with max error < 0.5 m.

5.3 Use Cases for Each Class

Need	Use
G2-continuous arc chain (compact, all tolerances met)	`G2ClothoidApproximator` + `G2ClothoidConfig`

Table 9: Choosing the right tool for the task.

6 Summary

The `clothoid.py` module fits a G2-continuous chain of clothoid arcs (Bertolazzi-Frego `SolveG2`) to a Kalman-filtered GPS track. Each interval between support points is solved to exact G2 continuity — position, tangent, and curvature are continuous at every joint. The greedy forward search guarantees max lateral error ≤ `position_tolerance_m` (default 0.5 m) with no rubber-band correction or post-processing required.

The critical distinction between these two: fitted κ serves **analysis** (derivatives, segment structure); geometric κ serves **reconstruction** (position recovery from support points).

References

[1] E. Bertolazzi and M. Frego, "G1 fitting with clothoids," *Mathematical Methods in the Applied Sciences*, vol. 38, no. 5, pp. 881–897, 2015, doi: 10.1002/mma.3114.

[2] H. Wild, *Strassenplanung*, 7th ed. Werner Verlag, 2010.

[3] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.

[4] E. Bertolazzi and M. Frego, "Interpolating clothoid splines with curvature continuity," *Mathematical Methods in the Applied Sciences*, vol. 41, no. 4, pp. 1723–1737, 2018, doi: 10.1002/mma.4700.

[5] E. Bertolazzi, "pyclothoids: Python bindings for the Bertolazzi-Frego clothoid library." [Online]. Available: https://github.com/ebertolazzi/Clothoids